

CSC9008 Cyber-Physical Systems

Shake it off: Checkpoint 2

Course Instructor: Prof. Chao Wang

Students: Xin-Shao Hon, Zhen-Rong Wu, Darrin Lin

Department of Computer Science and Information Engineering
National Taiwan Normal University

May. 19, 2026

Outline

- 1 System Architecture
- 2 MPU6050: Reading and Parsing Sensor Data
- 3 Complementary Filter for Attitude Estimation
- 4 PID Control System
- 5 Cable-Length Geometry for MG90S Servos
- 6 Live Demo

Project Overview

Goal

Build a self-stabilizing platform that keeps objects level despite external disturbances (manual tilting, vibration, earthquakes).

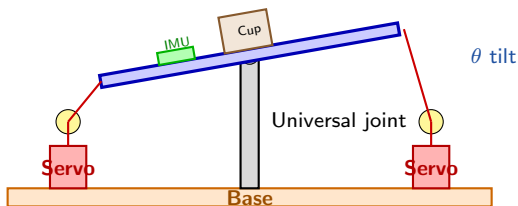
Hardware:

- STM32F401CCU6 (MCU)
- GY-521 / MPU6050 (IMU)
- 4 × MG90S servos
- Pulleys + strings
- Central universal joint

Control flow:

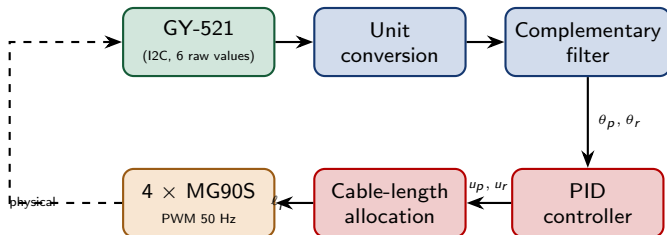
- Sense tilt via IMU
- Compute attitude (pitch, roll)
- PID computes correction
- Servos pull cables
- Platform tilts back to level

Mechanical Structure



- 4 cables pulling 4 platform corners (PA0 back, PA1 left, PA2 right, PA3 front)
- Cross-pairs work antagonistically: one pulls in, the other releases
- Central universal joint bears the load; servos only adjust angle

Signal Flow Diagram



- Closed-loop at **200 Hz** (5 ms control period)
- Pitch and roll axes controlled independently
- All computation runs on STM32F401, USB-CDC for logging

What the MPU6050 Measures

Two sensors in one chip

Accelerometer: measures *specific force* (in g)

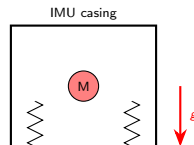
Gyroscope: measures *angular rate* (in deg/s)

Accelerometer (counter-intuitive!)

Even when stationary, the chip reads 1 g on z because:

$$\vec{a}_{\text{meas}} = \vec{a}_{\text{motion}} - \vec{g}$$

The spring inside resists gravity $\Rightarrow$ measurement is the reaction force.



I2C Communication and Raw Data Format

Each axis returns a **16-bit signed integer** (range $-32768 \sim +32767$). Six axes packed in 14 bytes starting at register 0x3B:

Reg	0x3B-3C	0x3D-3E	0x3F-40	0x41-42	0x43-44	0x45-46	0x47-48
Data	ACCEL_X	ACCEL_Y	ACCEL_Z	TEMP	GYRO_X	GYRO_Y	GYRO_Z

```
uint8_t buf[14];
HAL_I2C_Mem_Read(&hi2c1, 0xD0, 0x3B, 1, buf, 14, 100);
int16_t ax_raw = (int16_t)((buf[0] << 8) | buf[1]);
int16_t ay_raw = (int16_t)((buf[2] << 8) | buf[3]);
int16_t az_raw = (int16_t)((buf[4] << 8) | buf[5]);
int16_t gx_raw = (int16_t)((buf[8] << 8) | buf[9]);
int16_t gy_raw = (int16_t)((buf[10] << 8) | buf[11]);
int16_t gz_raw = (int16_t)((buf[12] << 8) | buf[13]);
```

Converting Raw Counts to Physical Units

Accelerometer ($\pm 2g$ range)

Full scale: $-32768 \leftrightarrow -2g$, $+32767 \leftrightarrow +2g$

$$1 \text{ raw unit} = \frac{2g - (-2g)}{65536} = \frac{1g}{16384}$$

$$a_{[g]} = \frac{a_{\text{raw}}}{16384}$$

Gyroscope ($\pm 250 \text{ deg/s}$ range)

$$1 \text{ raw unit} = \frac{500 \text{ deg/s}}{65536} \approx \frac{1 \text{ deg/s}}{131}$$

$$\omega_{[\text{deg/s}]} = \frac{\omega_{\text{raw}}}{131}$$

Sanity check (flat, stationary): $a_x \approx 0$, $a_y \approx 0$, $a_z \approx 1g$ (raw ≈ 16384).

Initialization and Calibration

Step 1: Wake up the chip (it boots in sleep mode!)

```
uint8_t d = 0x01; // X-gyro clock source, wake up
HAL_I2C_Mem_Write(&hi2c1, 0xD0, 0x6B, 1, &d, 1, 100);
// SMPLRT_DIV = 7 -> sample rate 125 Hz
// CONFIG      = 3 -> DLPF cutoff ~44 Hz (Fourier!)
// GYRO_CFG     = 0 -> +/- 250 deg/s
// ACCEL_CFG    = 0 -> +/- 2g
```

Step 2: Static calibration

Average $N = 1000$ samples at rest to estimate the constant bias:

$$a_{\text{off}} = \frac{1}{N} \sum_{k=1}^N a_k - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \omega_{\text{off}} = \frac{1}{N} \sum_{k=1}^N \omega_k$$

Every subsequent reading subtracts this offset:

$$a_{\text{corrected}} = \frac{a_{\text{raw}}}{16384} - a_{\text{off}}$$

Why Fuse Two Sensors?

	Accelerometer	Gyroscope
Steady state	Accurate (gravity)	Slow drift
Fast motion	Corrupted by inertia	Accurate
Long term	No drift	Drift grows with t
Good in	Low frequency	High frequency

Key Idea

Use accelerometer for low-frequency reference (gravity is reliable)

Use gyroscope for high-frequency dynamics (no motion artefacts)

Blend them with a **frequency-domain crossover** $\Rightarrow$ Complementary Filter

Kalman filter would adapt the weights dynamically, but the fixed-weight complementary filter is sufficient for our low-speed platform and is far cheaper computationally (1 multiply + 1 add per axis).

Attitude from Accelerometer Alone

Setup: Gravity in the world frame, z-axis up:

$$\vec{g}_W = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}$$

When the platform rotates by pitch θ_p (around y) and roll θ_r (around x), the IMU sees gravity rotated into its body frame.

Rotation matrix (passive, body-frame view)

$$R(\theta_p, \theta_r) = R_x(\theta_r) R_y(\theta_p)$$

$$\vec{g}_{\text{IMU}} = R \cdot \vec{g}_W$$

Since $\vec{g}_W$ has only a z-component, only the third column of R survives.
After multiplication:

$$a_x = \sin \theta_p$$

$$a_y = -\sin \theta_r \cos \theta_p \quad (\text{in units of } g)$$

$$a_z = \cos \theta_r \cos \theta_p$$

Deriving the Pitch/Roll Formulas

Combine $a_y^2 + a_z^2$ to isolate θ_p :

$$a_y^2 + a_z^2 = \sin^2 \theta_r \cos^2 \theta_p + \cos^2 \theta_r \cos^2 \theta_p = \cos^2 \theta_p$$

Therefore $\sqrt{a_y^2 + a_z^2} = \cos \theta_p$, and

$$\tan \theta_p = \frac{a_x}{\sqrt{a_y^2 + a_z^2}}$$

Final attitude equations (use atan2 for full-quadrant range)

$$\theta_p^{\text{acc}} = \text{atan2}\left(a_x, \sqrt{a_y^2 + a_z^2}\right)$$

$$\theta_r^{\text{acc}} = \text{atan2}(-a_y, a_z)$$

Verification (flat): $a_x=0, a_y=0, a_z=1 \Rightarrow \theta_p = \theta_r = 0$. ✓

Verification (pitch 30°): $a_x=0.5, a_z=0.866 \Rightarrow \theta_p = \text{atan2}(0.5, 0.866) = 30^\circ$. ✓

The Gyroscope Path

The gyroscope reports angular rate ω . To get an angle, integrate:

$$\theta^{\text{gyro}}(t) = \theta(0) + \int_0^t \omega(\tau) d\tau$$

Discrete form (sample every $\Delta t = 5 \text{ ms}$):

$$\theta_k^{\text{gyro}} = \theta_{k-1} + \omega_k \cdot \Delta t$$

Drift problem

Any constant bias b in $\omega_{\text{meas}} = \omega_{\text{true}} + b$ grows linearly:

$$\theta_k^{\text{gyro}} = \theta_{\text{true}}(t) + b \cdot t$$

A 0.05 deg/s bias becomes 3° after one minute.

- ⇒ Gyroscope alone diverges. Accelerometer alone is too noisy.
- ⇒ We need to fuse them.

The Complementary Filter

Discrete recursion

$$\hat{\theta}_k = \alpha \left(\hat{\theta}_{k-1} + \omega_k \Delta t \right) + (1 - \alpha) \theta_k^{\text{acc}}$$

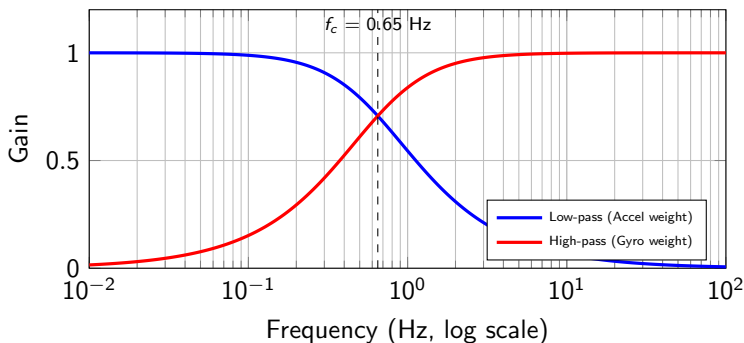
- $\alpha \approx 0.98$: trust gyroscope short-term, use accelerometer to slowly correct drift.
- Equivalent to: **high-pass on gyro + low-pass on accel**, summing to unity gain.

Crossover frequency (Fourier interpretation):

$$f_c = \frac{1 - \alpha}{2\pi \alpha \Delta t} = \frac{0.02}{2\pi \cdot 0.98 \cdot 0.005} \approx 0.65 \text{ Hz}$$

Below 0.65 Hz: accel dominates (gravity reference); above: gyro dominates (fast motion).

Filter Behavior (Conceptual)



- Sum of both curves equals 1 at every frequency $\Rightarrow$ *complementary*
- No phase distortion at the crossover
- One tuning parameter (α), trivial to implement

Implementation

```
#define ALPHA    0.98f
#define DT       0.005f    // 200 Hz control loop
#define R2D      (180.0f / 3.14159265f)

void Attitude_Update(Attitude_t *a, MPU6050_t *imu) {
    // Accel-based static angle
    float pitch_acc = atan2f(-imu->ax,
        sqrtf(imu->ay*imu->ay + imu->az*imu->az)) * R2D;
    float roll_acc  = atan2f(imu->ay, imu->az) * R2D;

    // Complementary filter
    a->pitch = ALPHA * (a->pitch + imu->gx * DT)
        + (1.0f - ALPHA) * pitch_acc;
    a->roll  = ALPHA * (a->roll  + imu->gy * DT)
        + (1.0f - ALPHA) * roll_acc;
}
```

Gyro-to-axis mapping (gx vs gy, and sign) is determined experimentally because it depends on how the IMU is physically oriented on the platform.

Control Problem Statement

Objective

Drive the platform attitude (θ_p, θ_r) to $(0, 0)$ in the presence of disturbances.

Define the error signal:

$$e(t) = \theta_{\text{ref}}(t) - \theta(t) = 0 - \theta(t) = -\theta(t)$$

We want a control law $u(t)$ (servo correction in degrees) that:

- Reacts quickly to disturbances
- Eliminates steady-state error (when the table itself is tilted)
- Avoids overshoot and oscillation

⇒ **PID controller**, the workhorse of feedback control (Week 11).

The Three Terms of PID

Continuous form

$$u(t) = \underbrace{K_p e(t)}_{\text{Proportional}} + \underbrace{K_i \int_0^t e(\tau) d\tau}_{\text{Integral}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{Derivative}}$$

Term	Reacts to	Cures	Side effect
P	Current error	Slow response	Steady-state error
I	Accumulated error	Steady-state error	Overshoot, slow
D	Rate of change of error	Overshoot, damping	Noise amplification

Analogy:

P = spring (pulls toward target), D = damper (resists motion), I = memory (accumulates over time)

Why I Eliminates Steady-State Error

Apply the **Final Value Theorem**:

$$e_{ss} = \lim_{t \rightarrow \infty} e(t) = \lim_{s \rightarrow 0} s E(s)$$

For a step disturbance $T_d(s) = T_0/s$ with PI(D) control:

$$e_{ss} = \lim_{s \rightarrow 0} \frac{s \cdot T_0/s}{1 + G_p(s)C(s)} = \frac{T_0}{1 + \lim_{s \rightarrow 0} G_p(s)C(s)}$$

Since $C(s)$ contains K_i/s , $\lim_{s \rightarrow 0} C(s) = \infty$, therefore:

$$e_{ss} = 0$$

Without the I term, $\lim_{s \rightarrow 0} C(s) = K_p$ is finite, yielding a non-zero steady-state error. This is the mathematical reason we need the integral term.

Discrete Implementation

Approximate the integral by rectangular sum, derivative by backward difference:

$$\int_0^{t_k} e \, d\tau \approx \sum_{i=0}^k e_i \Delta t, \quad \frac{de}{dt} \approx \frac{e_k - e_{k-1}}{\Delta t}$$

Discrete PID

$$u_k = K_p e_k + K_i \cdot \Delta t \sum_{i=0}^k e_i + K_d \cdot \frac{e_k - e_{k-1}}{\Delta t}$$

Two practical safeguards:

- **Anti-windup:** clip the integral so it cannot grow unboundedly during saturation
- **Output saturation:** clip u to the servo's physical range

Implementation in C

```
float PID_compute(float setpoint, float measured,
                  float *integral, float *prev_err,
                  float Kp, float Ki, float Kd, float dt)
{
    float err = setpoint - measured;

    // Integral term with anti-windup
    *integral += err * dt;
    if (*integral > I_LIMIT) *integral = I_LIMIT;
    if (*integral < -I_LIMIT) *integral = -I_LIMIT;

    // Derivative term
    float deriv = (err - *prev_err) / dt;
    *prev_err = err;

    // PID output with saturation
    float u = Kp*err + Ki>(*integral) + Kd*deriv;
    if (u > OUT_LIMIT) u = OUT_LIMIT;
    if (u < -OUT_LIMIT) u = -OUT_LIMIT;
    return u;
}
```

Two independent PID controllers run in parallel: one for pitch, one for roll.

From PID Output to Servo Action

PID outputs corrections u_p , u_r in **degrees of attitude correction**.
But each servo controls the **length of a cable** that pulls the platform.

Naive approach (does not work well)

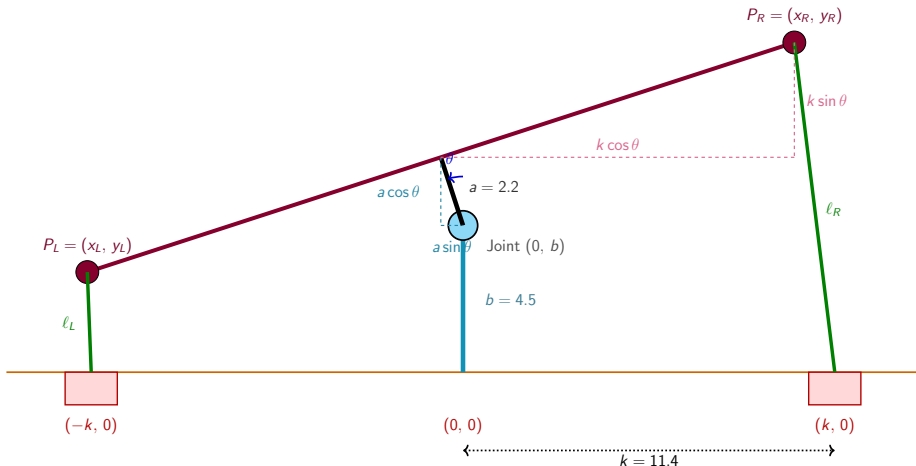
Map servo angle directly to PID output: $\theta_{\text{servo}} = 90^\circ + u$.

Problem: 1° of servo rotation does *not* produce 1° of platform tilt. The relationship is highly nonlinear because cables connect a rotating platform to fixed servo positions.

Our approach

Compute the *exact cable length* the platform would need at the desired tilt angle, then convert that length into a servo rotation via the spool radius.

Geometric Setup of the Mechanism



Platform rotates about the joint $(0, b)$. The upper pillar (length a) is rigidly attached and tilts with the platform. Cables connect platform ends to fixed servos at $(\pm k, 0)$.

Deriving the Cable Length (Step 1: Locate Platform Center)

The upper pillar rotates rigidly with the platform about the joint at $(0, b)$. The platform *center* is the top of the upper pillar. Before rotation it sits at $(0, b + a)$. The offset from the joint is $(0, a)$. After rotation by θ :

$$\text{offset}' = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} 0 \\ a \end{bmatrix} = \begin{bmatrix} -a \sin \theta \\ a \cos \theta \end{bmatrix}$$

Add back the joint position $(0, b)$:

$$P_{\text{center}} = (-a \sin \theta, b + a \cos \theta)$$

This term is what makes our model different from a naive single-pillar setup. Because a is not negligible compared to k , ignoring it would introduce a position error of up to $a \sin \theta_{\text{max}}$ in the platform center.

Deriving the Cable Length (Step 2: Platform Endpoints)

Each platform endpoint is offset $\pm k$ along the platform direction. After rotation, the offsets become $(\pm k \cos \theta, \pm k \sin \theta)$.

Add to the platform center P_{center} :

$$P_R = (-a \sin \theta + k \cos \theta, b + a \cos \theta + k \sin \theta)$$

$$P_L = (-a \sin \theta - k \cos \theta, b + a \cos \theta - k \sin \theta)$$

Cable length is the Euclidean distance from each endpoint to its servo:

$$\ell_R = \|P_R - (k, 0)\|, \quad \ell_L = \|P_L - (-k, 0)\|$$

Result

$$\ell_R(\theta) = \sqrt{(-a \sin \theta + k \cos \theta - k)^2 + (b + a \cos \theta + k \sin \theta)^2}$$

$$\ell_L(\theta) = \sqrt{(-a \sin \theta - k \cos \theta + k)^2 + (b + a \cos \theta - k \sin \theta)^2}$$

Sanity Check at $\theta = 0$

Substituting $\sin 0 = 0$, $\cos 0 = 1$:

$$\ell_R(0) = \sqrt{(0 + k - k)^2 + (b + a + 0)^2} = a + b$$

$$\ell_L(0) = \sqrt{(0 - k + k)^2 + (b + a - 0)^2} = a + b$$

Both cables have length $a + b$ at rest ✓

Cable length *change* relative to neutral position:

$$\Delta\ell_R(\theta) = \ell_R(\theta) - (a + b), \quad \Delta\ell_L(\theta) = \ell_L(\theta) - (a + b)$$

Numerical example: $a = 2.2$, $b = 4.5$, $k = 11.4$, $\theta = 10^\circ$

$$\ell_R(10^\circ) \approx 8.66 \text{ cm}, \quad \Delta\ell_R \approx +1.96 \text{ cm (release)}$$

$$\ell_L(10^\circ) \approx 4.69 \text{ cm}, \quad \Delta\ell_L \approx -2.01 \text{ cm (reel in)}$$

The two cables move *antagonistically* (one shortens, one lengthens) — exactly the behavior we want.

From Cable Length to Servo Angle

Each servo has a spool with circumference C . One full 360° revolution winds/unwinds one full circumference of cable. Therefore:

$$\Delta\ell = C \cdot \frac{\alpha}{360^\circ} \quad \Rightarrow \quad \boxed{\alpha = \frac{360^\circ \cdot \Delta\ell}{C}}$$

where α is the servo rotation (in degrees) and $\Delta\ell$ is the required cable-length change.

Final servo command (relative to the 90° neutral position):

$$\theta_{\text{servo}} = 90^\circ + \alpha$$

Continuing the example ($\Delta\ell_R = +1.96$ cm, $C = 2\pi r$ with $r = 1.0$ cm
 $\Rightarrow C \approx 6.28$ cm)

$$\alpha_R = \frac{360 \cdot 1.96}{6.28} \approx 112^\circ$$

A 112° servo rotation is required to release 1.96 cm of cable — which is why bigger spool circumference gives finer (but smaller-range) control.

Full Control Pipeline

- ① Read IMU, compute θ_p and θ_r via complementary filter.
- ② Run two independent PIDs:

$$u_p = \text{PID}_p(0 - \theta_p), \quad u_r = \text{PID}_r(0 - \theta_r)$$

- ③ For each axis, compute the required cable lengths using the geometric formulas:

$$\ell_R = \ell_R(u_p), \quad \ell_L = \ell_L(u_p)$$

- ④ Convert each cable length to a servo rotation:

$$\alpha_i = \frac{360^\circ \cdot (\ell_i - (a + b))}{C}$$

- ⑤ Output PWM via TIM2 channels (50 Hz, 1.0–2.0 ms pulse width).

Same procedure runs in parallel for the roll axis.

All computations complete within one 5 ms control period.